

Programming Lab 1 (CS21201)
M.Tech. I Semester (2026-27)
Assignment:- 03 (Part: B)

Q1. Build a Heap and Implement HeapSort

Build a heap and implement the HeapSort algorithm.

Algorithm: HeapSort(Tree T)

```
T.size = |nodes of T|
BuildHeap(T)

while T.size > 0 do
    DeleteMax(T)
```

Q2. Priority Queue Using a Red-Black Tree

Can a priority queue be implemented as a red-black tree? What advantages does a heap implementation have over a red-black tree implementation?

Q3. Priority Queue Using a 2D Array

Suppose we have a 2D array in which the following conditions are maintained for every (i, j) :

$$A(i, j) \leq A(i + 1, j)$$
$$A(i, j) \leq A(i, j + 1)$$

Can this 2D array be used to implement a priority queue? Justify your answer.

Red-Black Tree

A Red-Black Tree is a **self-balancing Binary Search Tree (BST)** in which each node has an additional color, either **Red or Black**. The colors follow certain rules to keep the tree approximately balanced after insertion and deletion.

Main properties:

1. Every node is either Red or Black.
2. The root node is always Black.
3. All NULL (leaf) nodes are considered Black.
4. A Red node cannot have a Red child.
5. Every path from a node to its NULL descendants contains the same number of Black nodes.

These properties ensure that the height of the tree remains **$O(\log n)$** . Therefore, searching, insertion, and deletion can be performed efficiently in **$O(\log n)$** time.

Red-Black Trees are commonly used in applications where frequent insertion, deletion, and searching operations are required.